

From Responses To Trajectories: Multi-Turn and Multi-Environment Reinforcement Learning

Kashif Rasul ([@krasul](#)) - Sergio Paniego ([@sergiopaniego](#))

PyTorch Conference Europe 2026

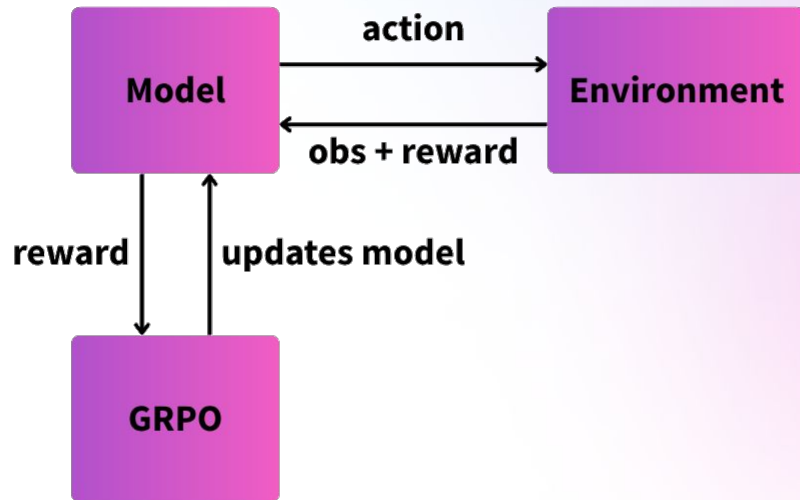
Machine Learning Engineers @ Hugging Face

From Static to Interactive

1. SFT: learn from demonstrations
prompt → completion
2. RLHF/DPO: learn from preferences
prompt → completion → preference
3. GRPO on static datasets: learn from reward signals (single turn)
prompt → completion → reward
4. **GRPO on trajectories: learn from interaction (multi-turn, envs)**
prompt → action → observation → action → ... → reward

What's needed for trajectory training

- Multi-turn generation loop (model $\leftrightarrow$ env)
- Environments that: reset, provide observations, and compute rewards
 - *Wordle, BrowserGym, CARLA simulator, ...*
- Correct loss masking (only train on model tokens)
- Scalable rollout generation 🏎️LLM

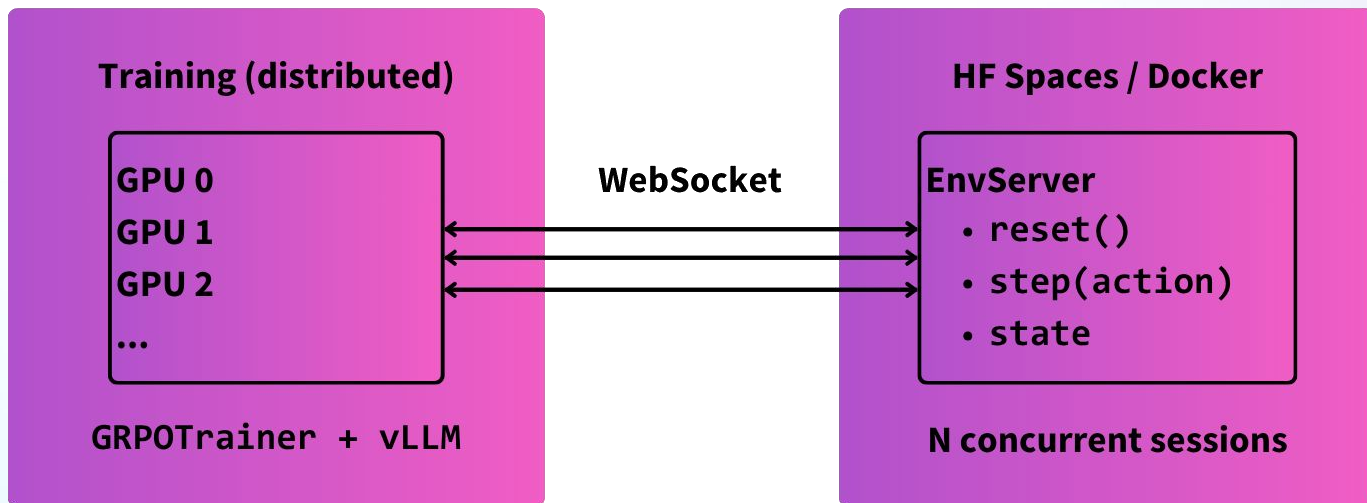


OpenEnv: a gym for LLMs

- Open source framework for creating, deploying and sharing RL envs
- Gymnasium API: `reset()` → `step(action)` → `state`
- Many envs: games, code, browsers, simulators...
- Deployable locally, in , or on  Spaces (`openenv push`)

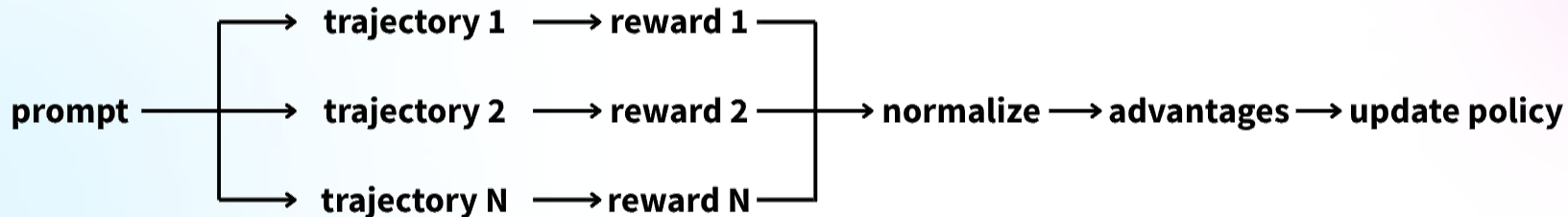
Wordle	CARLA	BrowserGym
Sudoku	Catch	...

OpenEnv architecture



TRL + GRPO recap

- GRPO gives the learning rule: group rewards, compute advantages, update the policy.
- Trajectory training keeps that objective, but replaces single completions with multi-turn rollouts.
- The hard part becomes rollout orchestration: environments, tools, masking, batching, and rewards.
- **TRL provides that orchestration layer.**



The `environment_factory` pattern

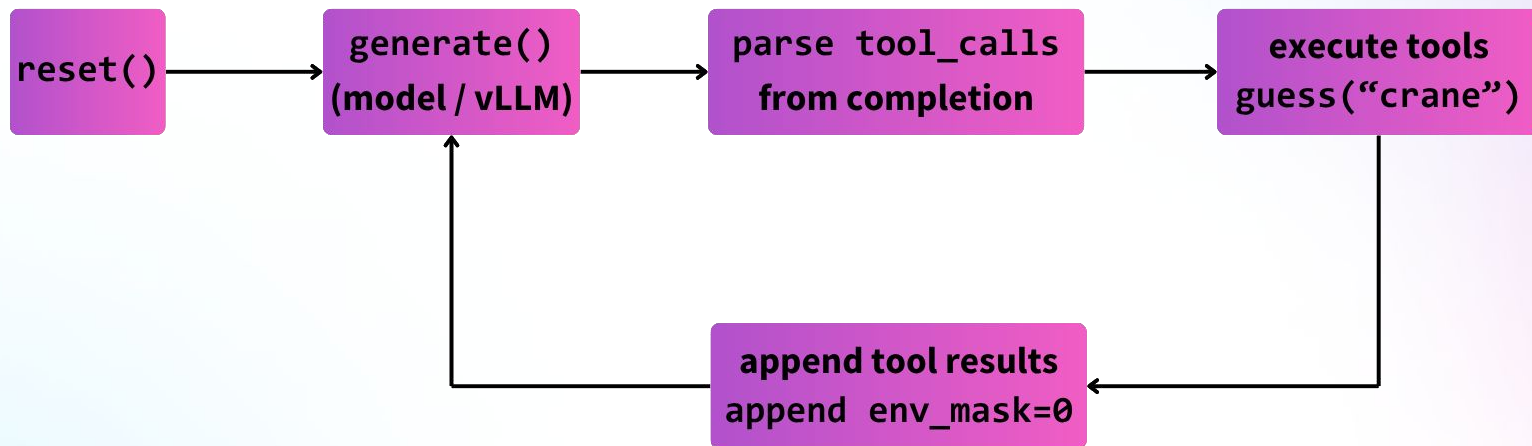
- `environment_factory` lets `GRPOTrainer` create one environment instance per generation and handle the multi-turn tool/environment loop automatically.
- Minimal contract:
 - `__init__()` sets up client/state
 - `reset(**kwargs)` starts a new episode and returns the initial observation
 - every public method besides `reset` becomes a tool
 - reward is stored on the environment instance
 - `reward_func(environments, **kwargs)` reads rewards back after rollout

```

1 class WordleEnv:
2     def __init__(self):
3         self.client = TextArenaEnv(base_url=env_url)
4         self.reward = 0.0
5
6     def reset(self, **kwargs) -> str | None:
7         result = self.client.reset()
8         self.reward = 0.0
9         return result.observation.messages[0].content
10
11    def guess(self, guess: str) -> str:
12        """Make a guess in the Wordle environment."""
13        result = self.client.step(TextArenaAction(message=guess))
14        self.reward = result.reward
15        return result.observation.messages[0].content
16
17    def reward_func(environments, **kwargs):
18        return [env.reward for env in environments]
19
20    trainer = GRPOTrainer(
21        model="Qwen/Qwen3-1.7B",
22        environment_factory=WordleEnv,
23        reward_funcs=reward_func,
24    )

```


Multi-turn generation loop



Loop until: no tool calls or max_tool_calling_iterations reached

Training on the right tokens: env_mask

Model: "I'll guess crane"

Tool result:



Model: "Then I'll try..."

env_mask = 1 1 1 1 1 env_mask = 0 0 0 0 0 env_mask = 1 1 1 1 1



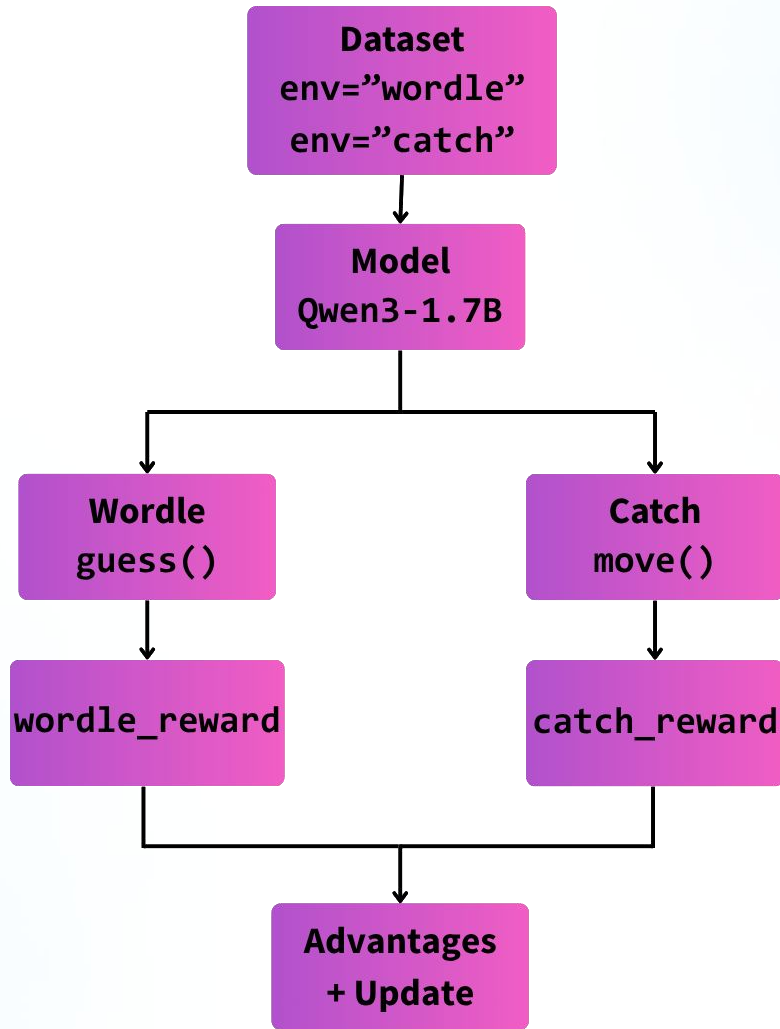
Model tokens (1) → compute loss



Environment tokens (0) → masked out

$$\text{loss_mask} = \text{completion_mask} * \text{tool_mask}$$

Multi-env training



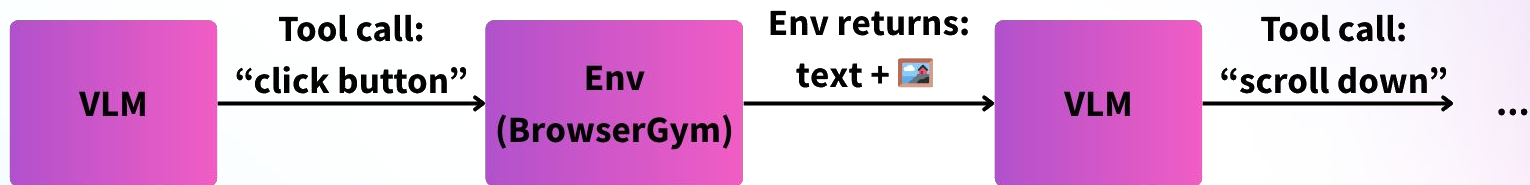
```

1 class MultiEnv:
2     def reset(self, **kwargs) -> str | None:
3         self.active = kwargs.get("env") # routed by dataset
4         ...
5
6     def guess(self, guess: str) -> str:
7         """Wordle tool"""
8         ...
9
10    def move(self, direction: str) -> str:
11        """Catch tool"""
12        ...
13
14    dataset = Dataset.from_dict({
15        "prompt": wordle_prompts + catch_prompts,
16        "env": ["wordle"] * n + ["catch"] * n,
17    })
18
19    trainer = GRPOTrainer(
20        model="Qwen/Qwen3-1.7B",
21        reward_funcs=[wordle_reward, catch_reward],
22        environment_factory=MultiEnv,
23    )

```

** Alternatively, `rollout_func` gives you full control over the generation pipeline*

Multimodal envs



```
1 # Tool responses can include images
2 {"role": "tool", "content": [
3     {"type": "text", "text": "Screenshot captured"},
4     {"type": "image", "image": <PIL.Image>}
5 ]}
```

CARLA: teaching LLMs to Drive

- [CARLA](#): autonomous driving simulator
- Scenario: vehicle heading towards pedestrians
- Model must observe, reason, and act to minimize harm
- Tools:
 - `observe()`: look at the scene
 - `emergency_stop()`: maximum braking
 - `lane_change(direction)`: swerve left/right
- Synchronous simulation
- Runs on HF Spaces, scalable with multiple instances

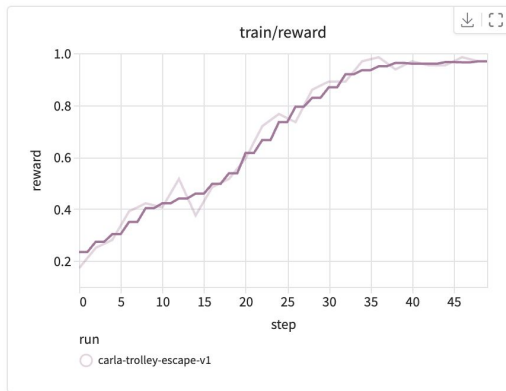


```

1 class CarlaGRPOEnv:
2     def __init__(self):
3         self.client = CarlaEnv(base_url=env_url)
4
5     def reset(self, **kwargs) -> str:
6         result=self.client.reset(scenario_name="trolley_micro_escape_exists")
7         self.reward = 0.0
8         return describe(result.observation)
9
10    def observe(self) -> str:
11        """Get the current scene description."""
12        ...
13
14    def emergency_stop(self) -> str:
15        """Apply maximum braking to stop the vehicle."""
16        ...
17
18    def lane_change(self, direction: str) -> str:
19        """Change lane. Direction: 'left' or 'right'."""
20        ...
21
22    trainer = GRPOTrainer(
23        model="Qwen/Qwen3-0.6B",
24        reward_funcs=reward_func,
25        environment_factory=CarlaGRPOEnv,
26    )

```

CARLA: from crashing to avoiding



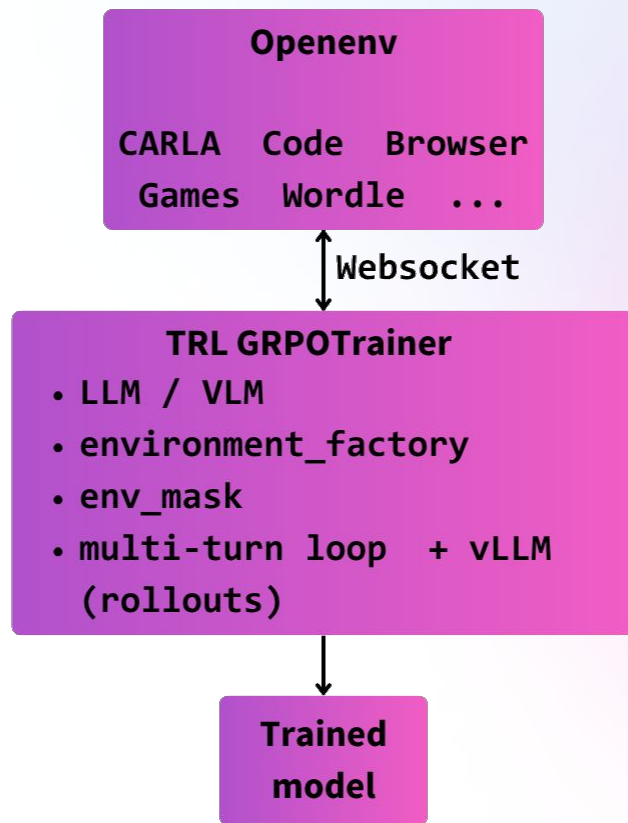
Qwen3-0.6B with GRPO

Also supported in VLMs
for Qwen3.5 or Gemma 4

- Step 1 – `observe()`
 - Speed: 39.6 km/h | 3 pedestrians at ~19m ahead
- Step 2 – `emergency_stop()` # start braking to reduce speed
 - Speed: 27.4 km/h | pedestrians at ~14m
- Step 3 – `lane_change("left")` # swerve while still slowing down
 - Speed: 27.6 km/h | pedestrians at ~10m (moving to adjacent lane)
- Step 4 – `emergency_stop()` # full stop to ensure no collision
 - Speed: 0.0 km/h | pedestrians at ~6m (car stopped, collision avoided ✓)

Putting it all together

- Multi-turn with `environment_factory`
- Multi-environment in one run
- Multimodal (VLMs + images)
- Many OpenEnv environments
- LLMs/VLMs learning to drive, play, browse, code...



What we're working on

- More environments
- AsyncGRPO
- Generate trajectories and share on the Hub
- Richer multimodal and agentic workflows

We're just getting started on envs 🤖



Thank you

Kashif Rasul ([@krasul](#)) - Sergio Paniego ([@sergiopaniego](#))

PyTorch Conference Europe 2026

Machine Learning Engineers @ Hugging Face